

LLM 推論與服務容量

模型運算、狀態記憶體與負載測試

一個 request 進來後，GPU 做了什麼？

當 request 變多，瓶頸出現在哪裡？

Qwen3 · Qwen3.5-122B-A10B · Qwen3.8-27B

vLLM serving / NVIDIA B200 180 GB

報告路線

同一個 request，從模型內部看到服務外部

01	LLM 與 vLLM 執行堆疊	10 分鐘
02	標準 Transformer 與 MoE	10 分鐘
03	Qwen 混合架構與 cache	15 分鐘
04	B200 記憶體與效能限制	5 分鐘
05	兩階段負載測試與容量	15 分鐘

Autoregressive inference

每次 forward 產生下一個 token 的分布



Prompt 的最後位置產生第一個 output token；下一輪再將這個 token 送入模型。

- 推論時更新 activation 與 request state，模型權重通常固定。
- 基準流程一次 decode 一個 token；speculative decoding 另外處理。

vLLM 的執行堆疊

API 到 GPU 的主要責任邊界

API server / frontend	HTTP、chat template、tokenization、串流回傳
Engine / scheduler	request 狀態、每輪 token budget、cache 配置
Executor / workers	分散式執行、各 GPU 的工作協調
Model runner	準備 tensors 與 metadata、graph、呼叫 model
Model / backend / kernels	模型 layer、attention / GDN / MoE、GPU 算子

Serve 不同模型，改動在哪裡？

Checkpoint、架構與硬體 backend 是不同層次的變更

層次	同架構的新 checkpoint	加入新架構
REST API	多半共用	多半共用；新增模態或輸出語意可能擴充
Tokenizer / parser	詞表、template、reasoning / tool parser	依模型格式調整
Model / loader	設定、權重、quantization metadata	新 module、weight mapping、parallel 分割
Runner / cache	通常沿用既有支援	視 state、metadata、graph 需求整合
GPU kernels	選用相容 backend	缺少算子或 layout 才新增 / 最佳化

Continuous batching

每輪調度重新組合仍在執行的 requests

Batching strategies for LLM inference

Individual requests



Dynamic batching



Continuous batching



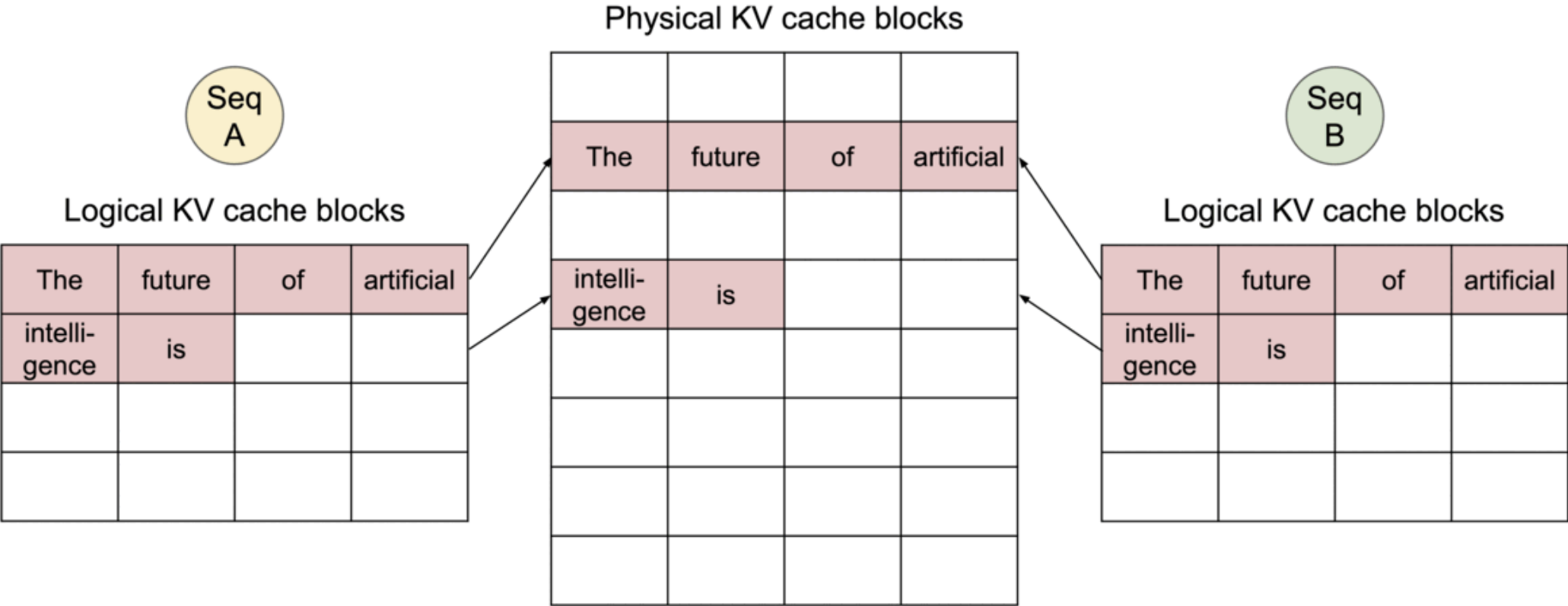
 baseten

使用者提供素材，圖上署名 Baseten。示意排程利用率，不代表等比例速度提升。

PagedAttention

Logical blocks 對應非連續的 physical KV blocks

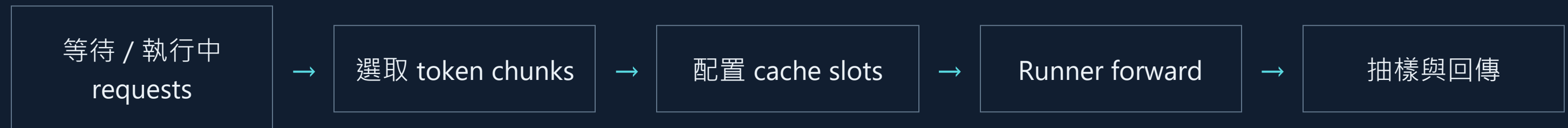
0. Shared prompt: Map logical blocks to the same physical blocks.



使用者提供 GIF，保留原始動畫。共享前綴及寫入分歧需維持正確的 block ownership。

每一輪 scheduler 做什麼？

以 token budget 與 cache 可用量決定本輪工作



- 新 request 可以進入 prefill。
- 長 prompt 可切成多個 prefill chunks。
- 執行中 request 繼續 decode。
- 資源不足可能等待或 preempt。

一輪 batch 可以包含 prefill tokens 與 decode tokens；它們的運算形狀與成本不同。

Qwen3-32B：標準 Transformer 基準

64 層 full attention，搭配 dense SwiGLU FFN



Hidden size	Q heads	KV heads	Head dim	FFN intermediate
5120	64	8	128	25,600

Prefill 與 decode

相同 layers，不同 token 維度與歷史資料存取

6 / 6

已知 prompt



Embedding → Decoder layers × L → LM head / sampling

生成 output



已存 KV 位置

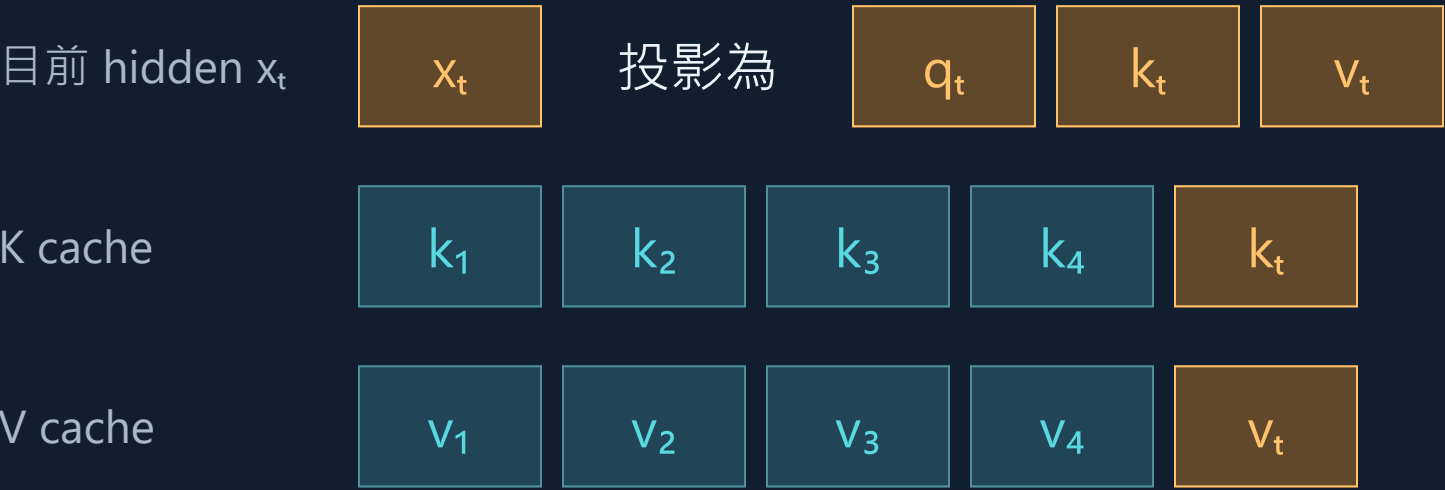
A B C D E F

抽樣 G。若此處結束，G 尚不需要再寫入 KV。

Decode 如何使用 KV cache

每層只新增目前位置的 K/V，再讀取歷史 K/V

5 / 5



$$o_t = \text{softmax}(q_t K^T / \sqrt{d}) V$$

Output projection 與 residual 後繼續下一層；Q 不需持久快取。

Prefill 與 decode 的效能形狀

瓶頸取決於 batch、context、kernel 與精度

	Prefill	Decode
輸入位置數	多個 prompt tokens	基準為每個 request 一個新 token
矩陣運算	大 GEMM，較易提高算術強度	小 batch 常受權重讀取與 launch 限制
Attention	長序列全注意力計算量快速增加	每步掃讀更長歷史 KV
服務觀察	影響 TTFT，可能干擾 decode	影響 ITL / TPOT 與輸出吞吐

高 concurrency 能攤平部分權重讀取成本，同時也可能拉長每個人的 token 間隔。

Dense 與 MoE 的差別

MoE 改變 FFN 的計算路徑

hidden x_t $\rightarrow$ router scores $\rightarrow$ Top-2 (8 個 experts 的教學示意)

E1 0.03	E2 0.04	E3 0.51	E4 0.03	E5 0.04	E6 0.05	E7 0.27	E8 0.03
------------	------------	------------	------------	------------	------------	------------	------------

$0.65 \times E3(x_t) + 0.35 \times E7(x_t) \rightarrow$ 合併輸出

下一個 token 或 layer 可以選不同 experts 。 Qwen122B 實際為 8 routed + 1 shared 。

MoE 的 activated parameters 與記憶體

少算一些 experts，仍需部署整組權重

122B

total parameters

10B

activated parameters / token

- 單個 token 只選部分 experts；同一個 batch 的 tokens 可能觸及更多 experts。
- 一般全 GPU 部署仍要存放所有 experts；offload 是另一項取捨。
- Expert parallelism 引入 token dispatch / combine 與負載不均問題。

Qwen3.5 之後的混合堆疊

Token mixer 與 FFN 類型分開比較

模型	Full attention	Gated DeltaNet	FFN
Qwen3-32B	64 層	無	Dense
Qwen3.5-122B-A10B	12 層	36 層	MoE
Qwen3.8-27B	16 層	48 層	Dense



混合模型每四層包含一層 full attention ；每個 mixer 後仍有 FFN 。

Gated DeltaNet 與 Mamba 的關係

共享 recurrent-state 思路，更新規則有所不同

Gated DeltaNet

- 輸入產生 q 、 k 、 v 與控制 gates。
- 以 decay 與 delta rule 更新 state。
- Qwen 混合模型使用這種 mixer。

Serving 的命名

- vLLM 將 GDN 放在 mamba 相關模組下。
- 設定可能出現 `mamba_ssm_dtype`。
- 本報告稱它為 GDN recurrent state，並對應常說的 SSM cache。

GDN 的一次 state update

固定大小矩陣承接歷史資訊

5 / 5



S_{t-1} : 舊 state

$$o_t = S_t q_t$$



S_t : 同樣的維度

用 q 讀出結果，再經 output gate / projection ; conv history 也要延續。

概念動畫，色彩只表示 state 更新。矩陣元素沒有實測或語意對應。

Hybrid prefill 與 decode

GDN state 和 full-attention KV 必須一起正確延續

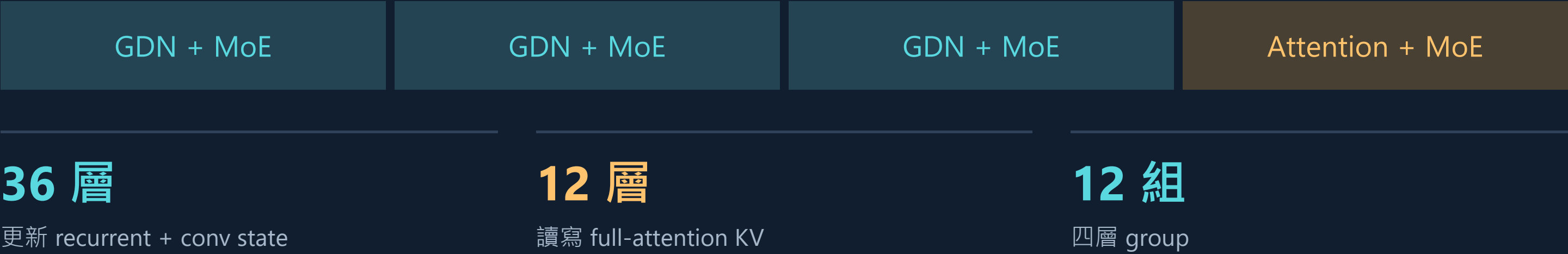
階段	GDN layers	Full-attention layers
Prefill	chunkwise 計算 prompt 並保留終點 state	因果 attention 並寫入 prompt K/V
Decode	讀取舊 state，更新為新 state	讀取歷史 KV，追加目前 K/V
保存	recurrent matrix + convolution state	每層、每位置的 K/V
隨 context 增長	最小 live state 大致固定	live KV 線性增長

GDN prefill 可以使用 chunkwise 平行演算法；不必逐 token 呼叫 Python 迴圈。

兩個 Qwen 模型的一次 decode

沿著所有 layers 前進，兩種 cache 都會更新

6 / 6



所有 groups 完成後，final norm / LM head 產生下一 token 的分布。

KV cache 與 recurrent state 的生命週期

Live state 與可重用 prefix 是不同的儲存需求



機制	需要保存什麼	注意事項
Full attention	前綴各位置 K/V	共享 block、引用與 eviction
GDN / SSM cache	對應前綴終點的 recurrent + conv state	只有最新 state，無法任意回到較早 prefix
Hybrid prefix hit	各類 layer 都能接續的共同位置	支援程度、checkpoint policy 依引擎版本
Speculative decode	可驗證與回復的狀態	額外 buffer、候選 token、commit / rollback

KV cache 的容量計算

先算每條 sequence，再考慮並行、分片與配置開銷

$$M_{KV} = 2 \times L_{\text{attention}} \times T \times H_{KV} \times d_{\text{head}} \times \text{bytes}$$

模型	KV heads × head dim	BF16 KV / token	T = 32,768
Qwen3-32B	8 × 128	256 KiB	8 GiB
Qwen3.5-122B-A10B	2 × 256	24 KiB	0.75 GiB
Qwen3.8-27B	4 × 256	64 KiB	2 GiB

模型全體、單條 sequence、無 prefix sharing。只計 full-attention KV；未含 GDN state、padding、runtime。1 GiB = 2³⁰ bytes。

SSM cache 的容量計算

GDN 的 recurrent matrix 與短 convolution state

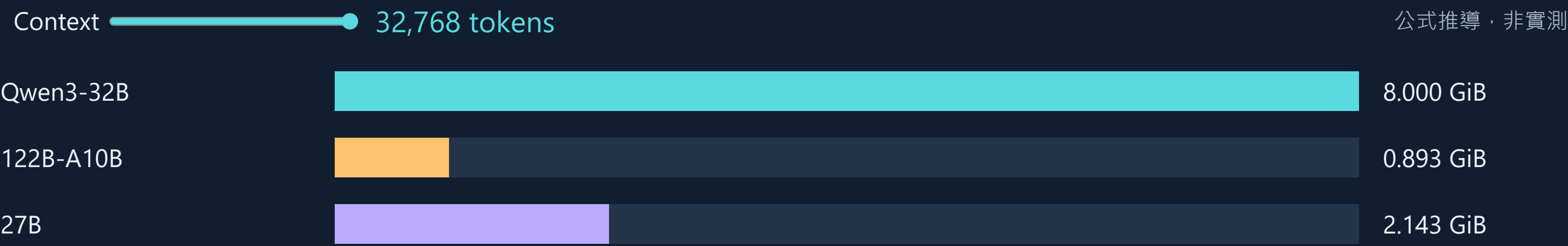
$$M_{\text{state}} = L_{\text{GDN}} \times H_V \times d_V \times d_K \times 4 \text{ bytes}$$

模型	FP32 recurrent matrices	BF16 conv history	合計最小 live state
122B-A10B	$36 \times 64 \times 128 \times 128 \times 4 = 144 \text{ MiB}$	約 2.53 MiB	約 146.53 MiB / sequence
27B	$48 \times 48 \times 128 \times 128 \times 4 = 144 \text{ MiB}$	約 2.81 MiB	約 146.81 MiB / sequence

並行序列增加時，state 仍會增加；「固定」只描述單序列對 context 長度的關係。

Context 長度與 cache 成長

在相同 BF16 KV、FP32 recurrent state 假設下比較



C × cache

多序列無共享時按數量增加

KV ∝ T

Full attention 歷史增長

State ∝ C

固定大小指的是每條序列

包含 BF16 KV、FP32 recurrent matrices、BF16 conv history。全模型合計，無分片、無 prefix sharing、無 speculative buffers。

B200 180 GB 的記憶體帳本

權重與 cache 必須共享 HBM 預算

模型	BF16 權重約值	1 byte / parameter 約值	單張 180 GB 的初步判讀
Qwen3-32B	64 GB	32 GB	仍需留 cache 與 runtime
Qwen3.8-27B	54 GB	27 GB	BF16 有空間，容量取決於 workload
Qwen3.5-122B-A10B	244 GB	122 GB	BF16 權重已超單卡；需分片或量化

$$\text{HBM} = \text{weights} + \text{KV} + \text{recurrent / conv state} + \text{activations} + \text{runtime} + \text{reserve}$$

GB 採 10⁹ bytes。以型號參數數量粗估，未計 scales、未量化層、vision / MTP 額外項及 runtime。不能當作載入成功保證。

GPU 資源與可觀察的症狀

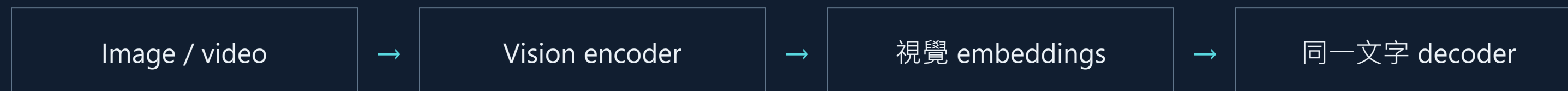
指標提供線索， profiler 與 queue / cache telemetry 才能定位

壓力來源	可能觀察到	下一步查證
權重讀取 / 小 batch	低 concurrency 的 TPOT 偏高	memory traffic、GEMM shape、launch gaps
長 context / cache	HBM 增加、attention 時間增加	KV 用量、context 分布、attention profile
Prefill 干擾 / 排隊	TTFT 或 ITL tail 上升	waiting requests、prefill chunks、GPU trace
MoE 分散式通訊	dispatch / combine 時間增加	expert load、collective timing、拓樸
Client / network / parser	client latency 與 server 指標分歧	送出時間、串流 chunk、CPU / network

ITL 先超標表示輸出節奏先失守；單靠 ITL 無法斷言 GPU decode 是唯一瓶頸。

多模態與 MTP 的位置

文字主線之外的額外工作



多模態

- 前處理、encoder 與視覺 tokens 增加成本。
- 文字 benchmark 無法直接代表影像 / 影片容量。

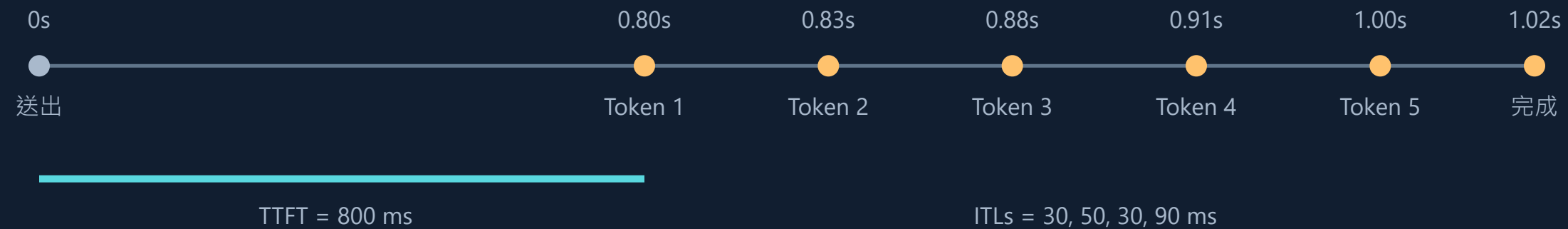
MTP / speculative decoding

- 候選 token 提案後，由 target model 驗證。
- 接受率、驗證成本與 state rollback 決定收益。

Latency 的時間邊界

Client 看到的 token 時間，包含服務外部成本

5 / 5



1.02 s

E2EL，至完成

55 ms

$TPOT = (1.02 - 0.80) / 4$

50 ms

四個 token intervals 的平均

工具與 backend 的計時邊界必須記錄，才能比較各次結果。

ITL、TPOT 與 percentile

先決定統計母體，再討論 p95 / p99

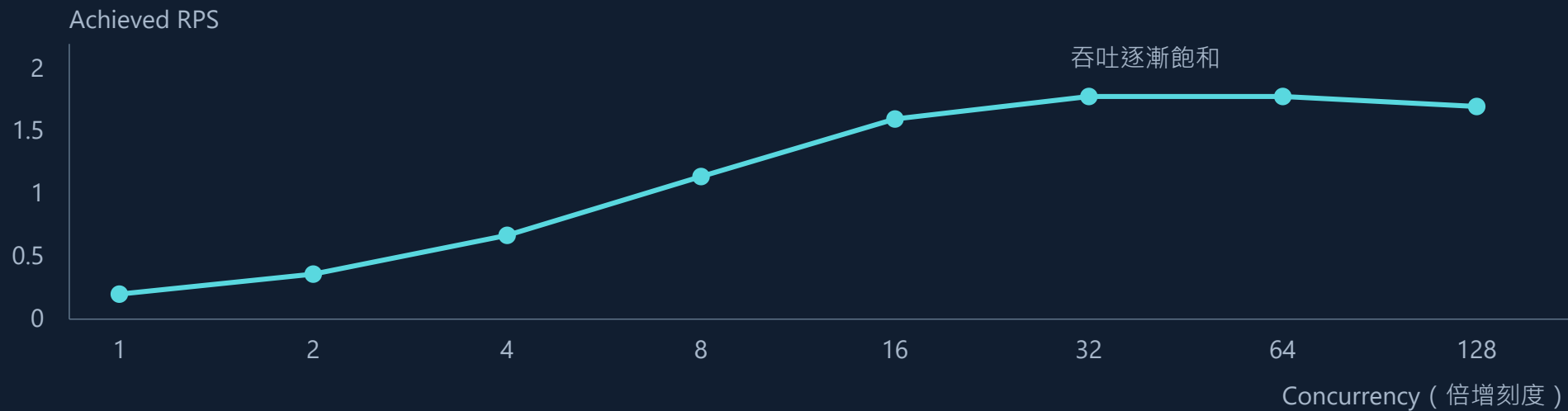
指標	每筆 request / interval	彙總方式
TTFT	$t_{\text{first}} - t_{\text{send}}$	每筆 request 一個樣本
ITL	$t_i - t_{(i-1)}$	各 token / stream chunk 間隔；確認 backend
TPOT	$(E2EL - TTFT) / (N_{\text{out}} - 1)$	每筆 request 的平均， $N_{\text{out}} > 1$
E2EL	$t_{\text{done}} - t_{\text{send}}$	包含完成語意的 client latency

所有 intervals 的平均，與「各 request 平均再平均」通常不同。p95 ITL 也不等於 p95 TPOT。

Stage 1 : Closed-loop saturation test

控制 concurrency，量測 achieved throughput

9 / 9



C = 32

18 s 平均 E2E，1.78 RPS

C = 64

36 s 平均 E2E，1.78 RPS

提高 concurrency 後，完成率可能幾乎不再增加，但 request 等待更久。

教學示意，非 B200 實測。C ≈ X × R 用於穩態一致性檢查。

Stage 2 : Open-loop SLO capacity test

控制到達率，觀察 queue 與 latency 是否穩定

到達率 2 req/s ▾

9 / 9

外界每秒 2 筆 → Queue → 單服務槽位每秒完成 1 筆

16

累積到達



8

累積完成

7

等待中 (另有至多 1 筆執行)

到達率超過處理率，等待隊列隨時間增長。完成後不會限制外界的新到達。

教學排隊示例。真實 Stage 2 需檢查 sent rate、queue trend、timeouts 與 tail latency。

Offered load、throughput 與 goodput

三種速率回答不同問題

同一觀測窗 $\Delta t = 100\text{ s}$	數量	速率
實際送出 requests	150	1.50 req/s
成功完成 requests	140	1.40 req/s
完成且符合每筆 SLO 的 requests	110	1.10 good req/s

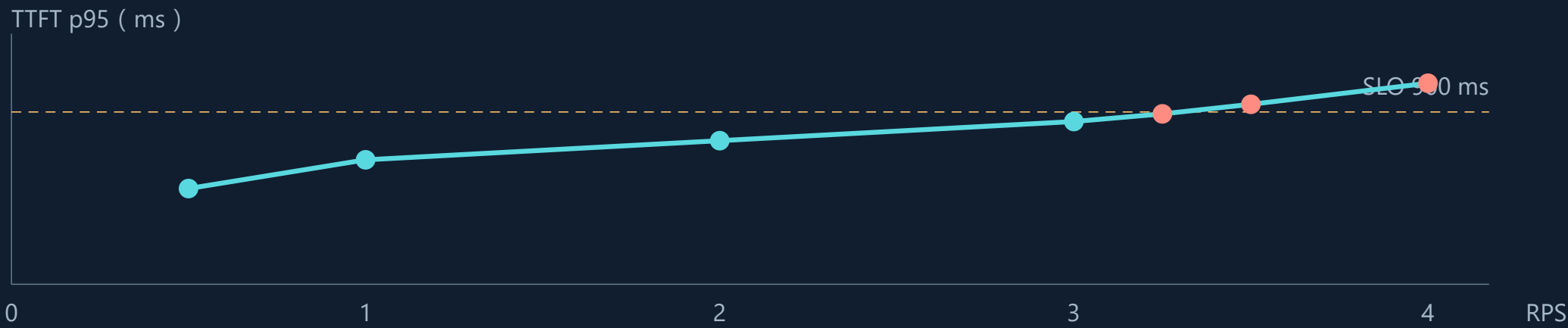
$$\text{Goodput} = \text{符合所選每筆 SLO 的成功 requests} / \Delta t$$

測試窗內尚未完成的 requests、失敗與 timeout，必須保留在報告中。

SLO 邊界搜尋

粗掃描範圍，再細掃並重複確認

7 / 7



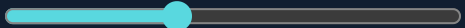
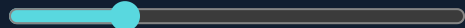
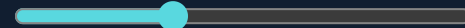
Offered RPS	TTFT mean ≤600	TTFT p95 ≤900	ITL mean ≤150	ITL p95 ≤300	All Pass
3.25	580 ms	890 ms	155 ms	305 ms	FAIL

3.25 RPS 的 TTFT 達標，但 ITL 失守。保留 3 RPS，再重複確認。

全部為教學示意數據。Run-level All Pass 另加 success / timeout / queue 穩定性條件。

由 RPS 換算服務人數

在穩定系統與一致邊界下使用平均值

完成率  平均 E2E  Think time 

$X = 8 \text{ req/s}$ $R = 8 \text{ s}$ $Z = 20 \text{ s}$

64

$C = X \times R$ ，平均 outstanding

224

$U = X \times (R + Z)$ ，active users

1120

假設尖峰 active ratio = 20%

前提：相同 workload 已通過 SLO 且系統穩定；每位 user 一次只有一筆 request。

若「40 秒」代表完整 request 週期， $U = X \times 40$ 。

若它是回答後的 think time， $U = X \times (R + 40)$ 。

以上是使用行為模型試算，沒有量測或保證這些 RPS 能通過 SLO。

負載測試報告的完整結論

Workload 與 SLO 定義決定容量數字的意義

在指定模型版本、GPU 拓樸、精度與 workload 下，
透過 concurrency sweep 找到效率曲線，
再以 arrival-rate sweep 找到穩定的 SLO 邊界。

- 模型架構決定每步運算，以及要保存的歷史狀態。
- vLLM 排程與 cache 管理決定如何將多筆工作放進 GPU。
- Production 容量再納入真實流量、burst、headroom 與故障情境。

附錄 A：測試設定與量測窗

可重現性比單一最高 tok/s 更有用

分類	必須記錄
模型與引擎	model revision、vLLM version / commit、container digest、backend
硬體與並行	GPU 型號 / 數量、TP / EP / DP / PP、互連、功率設定
精度與最佳化	weight / KV / state dtype、prefix cache、chunked prefill、MTP
Workload	ISL / OSL 分布、prompt prefix、thinking、stop / EOS、seed
測試與統計	warmup、arrival window、drain、樣本數、重跑次數、errors
Telemetry	scheduled / sent / completed、queue、cache、CPU/GPU traces

附錄 B：vLLM bench 命令範例

先核對安裝版本的 vllm bench serve --help

```
vllm bench serve \  
  --backend openai-chat --base-url http://HOST:8000 \  
  --endpoint /v1/chat/completions --model MODEL \  
  --dataset-name random --random-input-len 2048 \  
  --random-output-len 512 --num-prompts 1000 \  
  --percentile-metrics ttft,tpot,itl,e2el \  
  --metric-percentiles 50,95,99 --save-result
```

Stage 1 追加

```
--request-rate inf  
--max-concurrency 32
```

Stage 2 追加

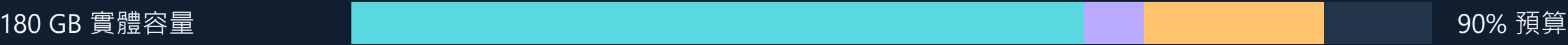
```
--request-rate 1.0  
--burstiness 1.0
```

附錄 C：記憶體配置試算

單卡教學預算，不含 allocator、分片與 kernel 的實際限制

122B-A10B / 1 byte weights ▾

Context 32,768 tokens



122 GB

粗估權重

10 GB

假設 runtime 預留

30 GB

可用於邏輯 cache

每條 cache ≈ 0.959 GB

$[30 / 0.959] = 31$

31 是這組簡化假設下的記憶體上界。SLO concurrency 需要另外測量。

固定 BF16 KV / FP32 recurrent state。排除量化 metadata、allocator padding、額外 checkpoints、vision / MTP 差異。不能作為部署承諾。

附錄 D：GDN 單一 head 的更新式

令 S 為 $d_v \times d_k$ 矩陣，省略 norm、conv 與 output gate

$$\bar{S}_t = \alpha_t S_{t-1}$$

$$r_t = v_t - \bar{S}_t k_t$$

$$S_t = \bar{S}_t + \beta_t r_t k_t^T \quad o_t = S_t q_t$$

α 控制衰減， β 控制修正幅度。這是 request state 的 forward update。

附錄 E：單 GPU 與多 GPU

TP、EP 與 DP 改變不同部分

策略	主要分配的內容	成本與限制
Tensor parallelism	同層矩陣與 attention heads	collectives、head 可分性、部分 KV 可能複製
Expert parallelism	不同 experts	token dispatch / combine、routing 不均
Data parallelism	完整模型 replicas	分流與 cache locality，每 replica 需權重
Pipeline parallelism	不同 layers	pipeline bubbles、跨 stage activation 傳輸

總 HBM 足夠只是起點；吞吐、尾延遲與跨卡通訊需要一起測。

附錄 F：來源與資料界線

查核日期 2026-09-08

[Qwen3-32B config](#)

[Qwen3.5-122B-A10B config](#)

[Qwen3.8-27B config](#)

[vLLM Architecture Overview](#)

[Model Runner V2](#)

[vLLM Qwen GDN implementation](#)

[Hybrid KV Cache Manager](#)

[vLLM benchmark 計算實作](#)

[使用者提供：計算 LLM 同時負載](#)

[vLLM PagedAttention](#)

[Qwen3.5-122B-A10B 模型卡](#)

[Qwen3.8-27B 模型卡](#)

[Qwen3.6-27B config](#)

[Inside vLLM](#)

[Gated Delta Networks 論文](#)

[vLLM state shape calculator](#)

[vLLM bench serve CLI](#)

[NVIDIA DGX B200 User Guide](#)

[Qwen3 Technical Report](#)

官方 config / 模型卡用於架構參數；容量由公式推導。吞吐曲線與動畫使用教學示例。圖片由使用者提供。